SECURITY ASSESSMENT REPORT

Classification: Confidential

Vulnerability Type: Insecure Direct Object Reference (IDOR) / Broken Object-Level Authorization (BOLA)

Target Category: Web Application Architecture / Document Generation Systems

Severity: High / Critical

1. Executive Summary

During a security architecture review of the document generation pipeline, a critical authorization flaw was identified within the PDF rendering functionality. The endpoint handles resource requests via unauthenticated, sequential parameters, presenting a textbook condition for **Insecure Direct Object Reference (IDOR)** (CWE-639).

Without robust server-side authorization validation, the application is highly susceptible to automated data harvesting. An unauthorized actor can programmatically iterate through parameters to compile a massive, structured database containing sensitive Personally Identifiable Information (PII) belonging to thousands of citizens.

2. Technical Vulnerability Analysis

The flaw resides in how the web application handles object lookups within user-supplied input variables.

Root Cause Analysis

The primary architectural failure stems from a reliance on **Security through Obscurity**. The system developers attempted to protect direct resource access by encoding integer record identifiers into Base64 format within the URL query string:

https://notunkurisports.gov.bd/generate_pdf.php?id=[BASE64_ENCODED_VALUE]

1. **Reversible Encoding:** Base64 is an encoding algorithm designed for data transport compatibility, not an encryption mechanism. It can be instantly reversed by any standard computing client without a key or secret.
2. **Predictable State Sequences:** The underlying data identifiers follow a strict, predictable sequential integer progression (e.g., 100001, 100002, 100003).
3. **Absence of Access Control Vectors:** The backend endpoint maps incoming parameters straight to database queries without checking if the client session has permission to view that specific record object.

3. Conceptual Flaw Demonstration (How It Functions)

Because this issue reflects a fundamental architectural omission rather than a complex technical exploit, the breakdown relies entirely on basic client-side parameter tampering:

Plaintext

Step 1: Resource Initialization
Authorized Client requests their legitimate document summary:
URL: https://notunkurisports.gov.bd/generate_pdf.php?id=MTAwMDAx
[Base64 Token "MTAwMDAx" -> Decodes to numerical ID: 100001]
Result: Server renders the PDF document corresponding to Account #100001.

Step 2: Logical Sequence Calculation
An external agent observes the encoding scheme and increments the integer sequence:
Target Integer: 100002
Encoding Execution: 100002 -> Base64 encoded -> "MTAwMDAy"

Step 3: Unauthorized Access Request

The agent resubmits the request using the calculated token:

URL: https://notunkurisports.gov.bd/generate_pdf.php?id=MTAwMDAy

Result: Because the application processes requests agnostically, it generates and serves the complete, verified PII of Account #100002 without verifying credentials.

4. Business Impact & Exposure Analysis

Mass Data Compilations

When an IDOR flaw is exposed via a public endpoint, the primary risk is **Bulk Data Harvesting (Scraping) at Scale**. A simple script utilizing browser automation engines can seamlessly iterate across thousands of records sequentially.

The specific data exposure of this vulnerability is unusually severe due to the structured layout of the target documentation. Threat actors can compile clean databases containing:

- **Full Legal Names and Demographic Details:** Bridging identities directly to institutional structures (e.g., schools, regional divisions).
- **Government Identifiers:** Exposing high-stakes indicators such as Birth Registration Numbers or National Identification Metrics.
- **Active Communication Channels:** Consolidating absolute home addresses and active mobile numbers.

Downstream Attack Vectors

Unlike fragmented data breaches, a fully populated, structured citizen database acts as a force-multiplier for secondary operations:

- **Hyper-Targeted Social Engineering (Phishing / Vishing):** Scammers can establish immediate, unearned authority by reading back highly

specific private details to a victim (such as parental names, exact registration codes, or school enrollment metrics) to execute financial fraud.

- **Identity Spoofing:** Exposed birth registration numbers and family identifiers satisfy common security verification baselines across public utility, health, and financial applications, undermining regional KYC (Know Your Customer) controls.

5. Defensive Mitigations

To definitively remediate this architectural gap, developers must implement structural protections on the server backend:

Phase 1: Implement Non-Sequential Identifiers (UUIDv4)

The absolute elimination of integer-predictability stops automated cycling tools instantly. Replace sequential record mappings with cryptographically random Universally Unique Identifiers (UUIDv4).

- **Secure Pattern:** /generate_pdf.php?id=b3c89f14-6d2a-4c80-a29d-14a5b23f87c1
- **Effect:** The calculation space becomes so vast (2^{128} states) that guessing a valid sequential link is mathematically impossible.

Phase 2: Enforce Server-Side Authorization Control

Every state-changing or data-rendering script must run access verification routines prior to parsing variables:

Plaintext

```
IF Session.IsAuthenticated() == FALSE THEN
    FORCE HTTP 401 Unauthorized
ELSE
    IF Session.User_ID !=
Database.GetOwnerOfRecord(Request.GetParam("id")) THEN
```

```
            FORCE HTTP 403 Forbidden
        ENDIF
ENDIF
```

Phase 3: Cryptographically Signed Temporary Links

If operational convenience dictates that field administrators or third parties must access the files cleanly via simplified mechanisms (such as scanning physical QR codes without manual credential pop-ups), utilize **Signed URLs**.

- Links are appended with a cryptographic signature hash (HMAC) calculated from the URL structure and a server-side secret key.
- The URL includes a strict expiration timestamp (e.g., valid for 10 minutes from generation), neutralizing long-term storage or data scraping sweeps.

6. Incident Recovery Protocols (Loss Control)

If an endpoint is found to have experienced data harvesting prior to patching, organizations must initiate an immediate data containment strategy:

1. **Network Infrastructure Logging Analysis:** Review proxy server, load balancer, and web server access logs to isolate historical volumetric anomalies. Identify specific source IP addresses, browser user-agent signatures, or routing blocks that pulled consecutive URL sequences within compressed periods.
2. **Endpoint Revocation:** Invalidate the exposed URL routing immediately. Take the script offline or inject an immediate blanket authentication requirement while the primary codebase is rebuilt.
3. **Stakeholder and Identity Exposure Notification:** Advise registered users of potential exposure. Issue targeted guidance cautioning affected citizens against incoming phone calls or text messages

requesting financial details or verification PINs, as scammers may utilize the leaked database to establish false identity credentials.

7. Institutional Lessons Learned

The occurrence of IDOR flaws points to structural gaps in the underlying software development lifecycle (SDLC) that must be addressed through long-term organizational changes:

- **Decouple Transport Formats from Data Privacy:** Security layers must assume that any client-side string, parameter, or encoding value can and will be modified by external users. Encoding data (Base64, Hex, URL encoding) must never be substituted for actual authorization checks.
- **Shift-Left Security Implementation:** Access control matrices should be defined during the design phase of an application rather than patched post-deployment. Developers must treat data objects as secure by default, explicitly carving out public access routes only when strictly necessary and completely stripped of sensitive PII.
- **Automated Architecture Auditing:** Integrate automated static application security testing (SAST) tools into compilation pipelines to identify code blocks that pull parameters straight into query variables without matching middleware validation checks.